

WEEK-5

AIM: Implement sending messages both synchronously and asynchronously in the producer.

Step 1 — Project directory

```
mkdir -p kafka-producer-demo\src\main\java\com\example\kafka
```

Step 2 — Project structure

```
kafka-producer-demo/
├── pom.xml                # Maven configuration file
├── src/
│   └── main/
│       └── java/
│           └── com/
│               └── example/
│                   └── kafka/
│                       ├── AsyncProducer.java    # Asynchronous producer class
│                       └── SyncProducer.java      # Synchronous producer class
├── target/
│   ├── classes/
│   └── kafka-producer-demo-1.0-SNAPSHOT-shaded.jar # Compiled classes and JARs
├── docker-compose.yml    # For Kafka & Zookeeper containers
└── lib/                  # Optional: external libraries
```

Step 3 — docker-compose.yml

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
      - 9093:9093
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Note: "kafka:9092" only resolves inside the Docker network. Any client running on the host machine (like the Java producer below) must connect through the PLAINTEXT_HOST listener — localhost:9093 / 127.0.0.1:9093 — NOT 9092.

Step 4 — Start containers

```
docker-compose up -d
```

Step 5 — pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example.kafka</groupId>
  <artifactId>kafka-producer-demo</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>Kafka Producer Demo</name>
  <description>A simple Kafka producer example using Java and Maven</description>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <kafka.version>3.6.0</kafka.version>
    <slf4j.version>2.0.9</slf4j.version>
  </properties>

  <dependencies>
    <!-- Kafka client library -->
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
      <version>${kafka.version}</version>
    </dependency>

    <!-- Simple logger -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>${slf4j.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Compiler plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
        <configuration>
```

```

        <source>${maven.compiler.source}</source>
        <target>${maven.compiler.target}</target>
    </configuration>
</plugin>

<!-- Shade plugin for runnable fat-jar -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.5.0</version>
    <executions>
        <execution>
            <phase>package</phase>
            <goals>
                <goal>shade</goal>
            </goals>
            <configuration>
                <createDependencyReducedPom>false</createDependencyReducedPom>
                <transformers>
                    <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                        <!-- Default entry point. Run SyncProducer separately
                             with -cp instead of rebuilding this. -->
                        <mainClass>com.example.kafka.AsyncProducer</mainClass>
                    </transformer>
                </transformers>
            </configuration>
        </execution>
    </executions>
</plugin>
</plugins>
</build>
</project>

```

Step 6 — AsyncProducer.java

```

package com.example.kafka;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;

import java.util.Properties;
import java.util.concurrent.ExecutionException;

public class AsyncProducer {
    public static void main(String[] args) {
        String topic = "demo-topic";

        // Kafka producer configuration
        Properties props = new Properties();
        // Use the HOST listener (9093), not the internal Docker listener (9092)
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9093");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.ACKS_CONFIG, "1"); // single broker ack
        props.put(ProducerConfig.RETRIES_CONFIG, 3);
    }
}

```

```

        props.put(ProducerConfig.LINGER_MS_CONFIG, 10); // small batching

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {

            // ===== ASYNCHRONOUS MESSAGES =====
            System.out.println("=== Sending ASYNCHRONOUS messages ===");
            for (int i = 1; i <= 5; i++) {
                String key = "async-key-" + i;
                String value = "async-message-" + i;

                ProducerRecord<String, String> record = new ProducerRecord<>(topic, key,
value);

                producer.send(record, (metadata, exception) -> {
                    if (exception != null) {
                        System.err.printf("Failed to send %s : %s%n", value,
exception.getMessage());
                    } else {
                        System.out.printf("Sent async: key=%s value=%s partition=%d
offset=%d%n",
                                key, value, metadata.partition(), metadata.offset());
                    }
                });
            }

            // flush to ensure all async messages are sent
            producer.flush();

            // ===== SYNCHRONOUS MESSAGES =====
            System.out.println("\n=== Sending SYNCHRONOUS messages ===");
            for (int i = 1; i <= 5; i++) {
                String key = "sync-key-" + i;
                String value = "sync-message-" + i;

                ProducerRecord<String, String> record = new ProducerRecord<>(topic, key,
value);

                try {
                    RecordMetadata metadata = producer.send(record).get(); // sync send
                    System.out.printf("Sent sync: key=%s value=%s partition=%d
offset=%d%n",
                                key, value, metadata.partition(), metadata.offset());
                } catch (InterruptedException | ExecutionException e) {
                    System.err.printf("Failed to send %s : %s%n", value, e.getMessage());
                }
            }

            System.out.println("\nAll messages sent successfully!");
        }
    }
}

```

Step 7 — SyncProducer.java

```

package com.example.kafka;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;

```

```

import java.util.Properties;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public class SyncProducer {
    public static void main(String[] args) {
        String topic = "demo-topic";

        // Kafka producer configuration
        Properties props = new Properties();
        // Use the HOST listener (9093), not the internal Docker listener (9092)
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9093");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.ACKS_CONFIG, "all"); // wait for full commit
        props.put(ProducerConfig.RETRIES_CONFIG, 3);
        props.put(ProducerConfig.LINGER_MS_CONFIG, 10); // optional small batching

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {

            System.out.println("=== Sending SYNCHRONOUS messages ===");

            for (int i = 1; i <= 10; i++) { // sending 10 messages
                String key = "sync-key-" + i;
                String value = "sync-message-" + i;
                ProducerRecord<String, String> record = new ProducerRecord<>(topic, key,
value);

                try {
                    // send() returns a Future<RecordMetadata> – BLOCKING call with timeout
                    RecordMetadata metadata = producer.send(record).get(5,
TimeUnit.SECONDS);
                    System.out.printf("Sent sync: key=%s value=%s partition=%d
offset=%d%n",
                                key, value, metadata.partition(), metadata.offset());
                } catch (InterruptedException | ExecutionException | TimeoutException e) {
                    System.err.printf("Failed to send record key=%s value=%s : %s%n", key,
value, e.getMessage());
                    // decide whether to continue sending or break
                }
            }

            System.out.println("All synchronous messages sent successfully!");

        } catch (Exception e) {
            System.err.println("Producer failed: " + e.getMessage());
        }
    }
}

```

Step 8 — Final execution steps

1. `mvn clean package`
2. `java -jar target/kafka-producer-demo-1.0-SNAPSHOT-shaded.jar`

```
(runs AsyncProducer – the mainClass set in the shade plugin manifest)
3. java -cp target/kafka-producer-demo-1.0-SNAPSHOT-shaded.jar
com.example.kafka.SyncProducer
(runs SyncProducer directly by fully-qualified class name, since only one
mainClass can be baked into the jar's manifest)
```

Note: There is only ONE shaded jar (kafka-producer-demo-1.0-SNAPSHOT-shaded.jar), named after the artifactId in pom.xml. There is no separate "async-producer-1.0-SNAPSHOT-shaded.jar" — that step in the original document referenced a jar that does not exist and has been removed.

Sample Output

```
9951599686@LAPTOP-446G38UQ MINGW64 ~/Desktop/Week-5/kafka-producer-demo (master)
$ java -jar target/kafka-producer-demo-1.0-SNAPSHOT-shaded.jar
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
=== Sending ASYNCHRONOUS messages ===
Sent async: key=async-key-1 value=async-message-1 partition=0 offset=20
Sent async: key=async-key-2 value=async-message-2 partition=0 offset=21
Sent async: key=async-key-3 value=async-message-3 partition=0 offset=22
Sent async: key=async-key-4 value=async-message-4 partition=0 offset=23
Sent async: key=async-key-5 value=async-message-5 partition=0 offset=24

=== Sending SYNCHRONOUS messages ===
Sent sync: key=sync-key-1 value=sync-message-1 partition=0 offset=25
Sent sync: key=sync-key-2 value=sync-message-2 partition=0 offset=26
Sent sync: key=sync-key-3 value=sync-message-3 partition=0 offset=27
Sent sync: key=sync-key-4 value=sync-message-4 partition=0 offset=28
Sent sync: key=sync-key-5 value=sync-message-5 partition=0 offset=29

All messages sent successfully!
```

Summary of corrections made from the original document

1. BOOTSTRAP_SERVERS_CONFIG changed from "127.0.0.1:9092" to "127.0.0.1:9093" in both AsyncProducer.java and SyncProducer.java, since port 9092 is only advertised as "kafka:9092" (resolvable inside the Docker network only). Port 9093 is the PLAINTEXT_HOST listener meant for host-machine clients.
2. Removed redundant "mvn clean" and "mvn clean compile" steps — "mvn clean package" alone performs all needed build phases.
3. Removed the invalid step referencing a nonexistent "async-producer-1.0-SNAPSHOT-shaded.jar" file.
4. Added the correct command to run SyncProducer.java directly via -cp with its fully qualified class name, since the shaded jar's manifest can only declare one Main-Class (AsyncProducer).
5. Removed the "Dependency-reduced-pom.xml" step since createDependencyReducedPom is set to false in pom.xml, so Maven does not generate that file — saving it as a separate step was unnecessary/misleading.